

Bangladesh Agricultural University

Mymensingh – 2202, Bangladesh

Computer Science and Mathematics

Faculty of Agricultural Engineering and Technology

Hand-Written Tokenizers for Three Small Languages

LAB ASSIGNMENT REPORT

CSM 3222: Compiler

Submitted To

Mr. Raihan Kabir Rifat

Lecturer

Computer Science and Mathematics

Bangladesh Agricultural University

Submitted By

Md Asifuzzaman

ID: 2309005

Level-03 / Semester-02, 2022-23

Computer Science and Mathematics

Bangladesh Agricultural University

Date of Submission: October 27, 2026

Hand-Written Tokenizers for Three Small Languages

CSM 3222: Compiler

MD ASIFUZZAMAN, Bangladesh Agricultural University, Bangladesh

1 Introduction

This assignment implements three hand-written C++ tokenizers of increasing difficulty: arithmetic tokens (Task 1), parenthesis-free statements (Task 2), and literals with comments (Task 3). Each program scans the input file character by character with maximal munch and prints tokens followed by errors. Each task is described by its design, buffering and positions, error recovery, and test results.

2 Task 1: Arithmetic Tokens

2.1 Design

Tokens: `INT`, `FLOAT` and the single-character operators `+ - * / % ^ ( )`. Only numbers need lookahead: after the digits, a `.` is part of a `FLOAT` only if a digit follows it. Otherwise the dot is an invalid character (`3.14.5`, `.5`).

2.2 Buffering and Positions

Each line is read into a buffer. `advance()` consumes characters and `peek()` looks at the next one without consuming it. The line counter increases per line, and the column is the start index of the lexeme plus one. Tabs count as one column and `\r` is whitespace.

2.3 Error Recovery

Letters, `=`, `.`, `$$` and other symbols are recorded as `INVALID_CHAR` at their start position. A run of invalid symbols is one error. Scanning then continues, so errors never stop the tokenizer.

2.4 Test Results

```
#include <iostream>
#include <cstring>
#include <string>
#include <fstream>
#include <vector>
#include <map>

using namespace std;

typedef struct invalid_char
{
    int row;
    int col;
    string ch;
} inv;


map<string, string> op = {
```

Author's Contact Information: Md Asifuzzaman, Bangladesh Agricultural University, Department of Computer Science and Mathematics, Mymensingh, Bangladesh, asifuzzaman.2309005@bau.edu.bd.

```
    {"+", "PLUS"},
    {"-", "MINUS"},
    {"*", "STAR"},
    {"/", "SLASH"},
    {"%", "PERCENT"},
    {"^", "CARET"},
    {"(", "LPAREN"},
    {")", "RPAREN"},
    {"0", "INT"},
    {"1", "INT"},
    {"2", "INT"},
    {"3", "INT"},
    {"4", "INT"},
    {"5", "INT"},
    {"6", "INT"},
    {"7", "INT"},
    {"8", "INT"},
    {"9", "INT"},
};;


void skipws(string &prog, int &start)
{
    while (start < prog.length() && isspace(prog[start]))
    {
        ++start;
    }
}

string advance(string &prog, int &start, int count = 1)
{
    int pos = start;
    start += count;
    return prog.substr(pos, count);
}

char peek(string &prog, int &start)
{
    return prog[start + 1];
}

string readString(string &prog, int &start)
{
    string str = "";
    if (prog[start] != '"')
    {
        cerr << "Error: string does not start with '\"'\n";
        return "";
    }
    advance(prog, start); // read the first "
```

```
    while (start < prog.length() && prog[start] != '"')
    {
        str += advance(prog, start);
    }

    if (start >= prog.length())
    {
        cerr << "Error: unterminated string\n";
        return "";
    }

    advance(prog, start); /// read ending "
    return str;
}

int main(int argc, char *argv[])
{
    fstream inf(argv[1], ios::in);
    if (!inf.is_open())
    {
        return 0;
    }

    string prog;
    string token_name;


    int start = 0;
    string token = "";

    vector<inv> inv_char;

    int row = 0;
    int col = 1;
    while (getline(inf, prog))
    {
        token_name = "";
        start = 0;
        col = 1;

        row++;
        while (start < prog.length())
        {
            skipws(prog, start);

            switch (prog[start])
            {
            case '"':
                col = start +1;
```

```
        token = readString(prog, start);
        break;
    case '{':
    case '}':
    case '(':
    case ')':
    case '[':
    case ']':
    case ';':
    case '+':
    case '-':
    case '%':
    case '/':
    case '^':
    case '*':
        col=start+1;
        token_name = op[token];
        token += advance(prog, start);
        token_name = op[token];
        break;
    case '.':
        col=start+1;
        inv_char.push_back({row, start, "."});
        start++;
        break;
    case '=':
        col=start+1;
        inv_char.push_back({row, start, "="});
        start++;
        break;
    case '$':
        col=start+1;
        if (peek(prog, start) == '$') {
            inv_char.push_back({row, start, "$$"});
            start++;
        }
        else {
            inv_char.push_back({row, start, "$"});
        }
        start++;
        break;
    default:
        col=start+1;
        if (isalpha(prog[start]))
        {
            char curr = prog[start];

            inv_char.push_back({row, start, {curr}});
            ++start;
```

```
            }
            else if (isdigit(prog[start]))
            {
                int temp = 0;
                while (start < prog.length() && isdigit(prog[start]))
                {
                    token += advance(prog, start);
                    token_name = "INT";
                }
                if (start < prog.length() && prog[start] == '.')
                {
                    if (start + 1 < prog.length() && isdigit(peek(prog, start)))
                    {
                        token += advance(prog, start); // min 1
                        while (start < prog.length() && isdigit(prog[start]))
                        {
                            token+=advance(prog, start);
                        }
                    }
                    token_name="FLOAT";
                }

                if (start < prog.length() && prog[start] == '.') {
                    inv_char.push_back({row, start, "."});
                    start++;
                }
            }
            else
            {
                while (start < prog.length() && !isspace(prog[start]) && !isalpha(
                    prog[start]))
                {
                    token += advance(prog, start);
                }
                inv_char.push_back({row, start, token});
                token.clear();
            }
            break;
        }

        if (token != "") cout << row << ":" << col << "␣" << token_name << "␣" <<
            token << '\n';


        token.clear();
        token_name.clear();
    }

}
```

```
    cout << "---␣ERRORS␣---\n";
    if (inv_char.size() == 0) cout << "(none)\n";
    else {
        for (auto it: inv_char) {
           cout << it.row << ":" << it.col+1 << "␣INVALID_CHAR" << "␣[" << it.ch << "]␣\
               n";
        }
    }

    cout << "---␣end␣---\n";
    return 0;
}
```

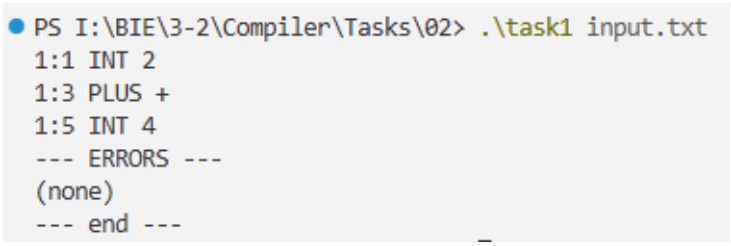


Fig. 1. Test 1.1

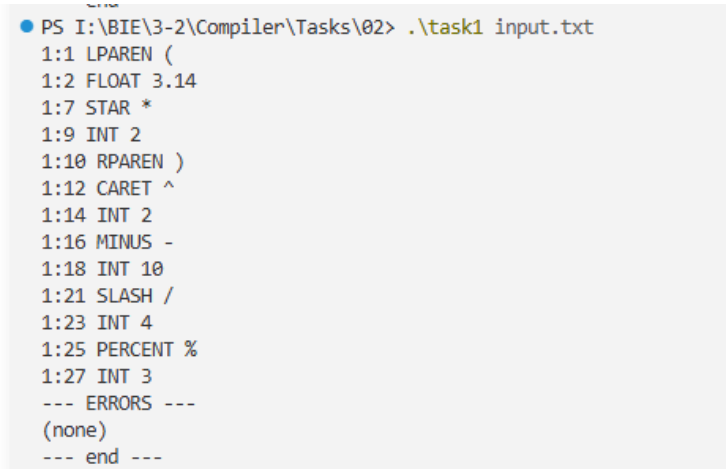


Fig. 2. Test 1.2

```
PS I:\BIE\3-2\Compiler\Tasks\02> .\task1 input.txt
1:1 INT 12
1:4 PLUS +
2:3 INT 007
2:6 STAR *
2:7 LPAREN (
2:8 INT 8
2:9 RPAREN )
--- ERRORS ---
(none)
--- end ---
```

Fig. 3. Test 1.3

```
PS I:\BIE\3-2\Compiler\Tasks\02> .\task1 input.txt
1:1 FLOAT 3.14
1:6 INT 5
1:8 PLUS +
1:11 INT 5
--- ERRORS ---
1:5 INVALID_CHAR [.]
1:10 INVALID_CHAR [.]
--- end ---
```

Fig. 4. Test 1.4

```
PS I:\BIE\3-2\Compiler\Tasks\02> .\task1 input.txt
1:1 LPAREN (
1:2 INT 1
1:4 PLUS +
1:7 INT 2
1:8 RPAREN )
1:10 STAR *
1:12 FLOAT 3
2:5 INT 4
2:7 SLASH /
2:8 SLASH /
2:10 INT 2
--- ERRORS ---
1:6 INVALID_CHAR [x]
1:13 INVALID_CHAR [.]
2:1 INVALID_CHAR [y]
2:3 INVALID_CHAR [=]
2:12 INVALID_CHAR [$$]
--- end ---
```

Fig. 5. Test 1.5

3 Task 2: Parenthesis-Free Statements

3.1 Design

An identifier is read completely and then looked up in a keyword set, so `iff` and `While` are identifiers. After `=`, `!`, `<`, `>` the scanner peeks for `=` to form `==`, `!=`, `<=`, `>=`, giving `===` → `EQ ASSIGN`. `&&` and `||` need a second identical character.

3.2 Buffering and Positions

Same reader as Task 1. The column is saved at the first character of each lexeme before it is consumed.

3.3 Error Recovery

A single `&` or `|` is an `INCOMPLETE_OPERATOR`; only that character is skipped. Other bad characters, including a digit not inside an identifier, form one `INVALID_CHAR` run that stops at whitespace or a letter (`9lives` → error `9`, then `lives`).

3.4 Test Results

```
#include <iostream>
#include <cstring>
#include <string>
#include <fstream>
#include <vector>
#include <map>
#include <set>

using namespace std;

typedef struct invalid_char
{
    int row;
    int col;
    string ch;
    string name;
} inv;

set<string> keywords = {
    "if", "else", "while", "for", "return", "int", "float"};

map<string, string> op = {
    {"=", "ASSIGN"},
    {"==", "EQ"},
    {"!=", "NEQ"},
    {"<=", "LE"},
    {">=", "GE"},
    {"<", "LT"},
    {">", "GT"},
    {"!", "NOT"},
    {"&&", "AND"},
    {"||", "OR"},
    {";", "SEMI"},
```

```
    {",", "COMMA"},
    {"{", "LBRACE"},
    {"}", "RBRACE"},
    {"[", "LBRACKET"},
    {"]", "RBRACKET"},
};

void skipws(string &prog, int &start)
{
    while (start < prog.length() && isspace(prog[start]))
    {
        ++start;
    }
}

string advance(string &prog, int &start, int count = 1)
{
    int pos = start;
    start += count;
    return prog.substr(pos, count);
}

char peek(string &prog, int &start)
{
    return prog[start + 1];
}

string readString(string &prog, int &start)
{
    string str = "";
    if (prog[start] != '"')
    {
        cerr << "Error: string does not start with '\"'\n";
        return "";
    }
    advance(prog, start); // read the first "

    while (start < prog.length() && prog[start] != '"')
    {
        str += advance(prog, start);
    }

    if (start >= prog.length())
    {
        cerr << "Error: unterminated string\n";
        return "";
    }

    advance(prog, start); /// read ending "
    return str;
```

```
}

int main(int argc, char *argv[])
{
    fstream inf(argv[1], ios::in);
    if (!inf.is_open())
    {
        return 0;
    }

    string prog;
    string token_name;

    int start = 0;
    string token = "";

    vector<inv> inv_char;

    int row = 0;
    int col = 1;
    while (getline(inf, prog))
    {
        token_name = "";
        start = 0;
        col = 1;

        row++;
        while (start < prog.length())
        {
            skipws(prog, start);

            switch (prog[start])
            {
            case '"':
                col = start + 1;
                token = readString(prog, start);
                break;
            case '{':
            case '}':
            case '(':
            case ')':
            case '[':
            case ']':
            case ';':
            case ',':
                col = start + 1;
                token += advance(prog, start);
                token_name = op[token];
                break;
            case '<':
```

```
case '>':
case '!':
case '=':
    col = start + 1;
    if (peek(prog, start) == '=')
    {
        token += advance(prog, start, 2);
    }
    else
    {
        token += advance(prog, start);
    }
    token_name = op[token];
    break;
case '&':
case '|':
    col = start + 1;
    if (peek(prog, start) == prog[start])
    {
        token += advance(prog, start, 2);
    }
    else
    {
        inv_char.push_back({row, start, string(1, prog[start]), "
            INCOMPLETE_OPERATOR"});
        start++;
    }
    token_name = op[token];
    break;
default:
    col = start + 1;
    if (isalpha(prog[start]) || prog[start] == '_')
    {
        while (start < prog.length() && (isalpha(prog[start]) || prog[start]
             == '_' || (prog[start] >= '0' && prog[start] <= '9')))
        {
            token += advance(prog, start);
            token_name = "IDENT";
        }
        if (keywords.find(token) != keywords.end())
        {
            token_name = "KEYWORD";
        }
    }
    else
    {
        while (start < prog.length() && !isspace(prog[start]) && !isalpha(
            prog[start]))
        {
            token += advance(prog, start);
```

```
                }
                inv_char.push_back({row, start, token, "INVALID_CHAR"});
                token.clear();
            }
            break;
        }

        if (token != "")
            cout << row << ":" << col << "␣" << token_name << "␣" << token << '\n';

        token.clear();
        token_name.clear();
    }
}

cout << "---␣ERRORS␣---\n";
if (inv_char.size() == 0)
    cout << "(none)\n";
else
{
    for (auto it : inv_char)
    {
        cout << it.row << ":" << it.col + 1 << "␣" << it.name << "␣[" << it.ch << "]
            ␣\n";
    }
}

cout << "---␣end␣---\n";
return 0;
}
```

```
PS I:\BIE\3-2\Compiler\Tasks\02> g++ -o task2 task2.cpp
PS I:\BIE\3-2\Compiler\Tasks\02> .\task2 input.txt
1:1  (
1:6 IDENT x2
1:8  )
2:1 IDENT y
2:3 ASSIGN =
--- ERRORS ---
1:3 INVALID_CHAR [1]
1:5 INVALID_CHAR [+]
1:11 INVALID_CHAR [*]
1:14 INVALID_CHAR [3.]
2:6 INVALID_CHAR [4]
2:9 INVALID_CHAR [//]
2:11 INVALID_CHAR [2]
2:14 INVALID_CHAR [$$]
--- end ---
```

Fig. 6. Test 2.1

```
PS I:\BIE\3-2\Compiler\Tasks\02> .\task2 input.txt
1:1 KEYWORD int
1:5 IDENT integer
1:12 COMMA ,
1:14 IDENT floats_2
1:22 COMMA ,
1:24 IDENT _x
1:26 SEMI ;
2:1 IDENT While
2:7 KEYWORD while
2:13 IDENT For
2:17 KEYWORD for
--- ERRORS ---
(none)
--- end ---
```

Fig. 7. Test 2.2

```
PS I:\BIE\3-2\Compiler\Tasks\02> .\task2 input.txt
1:1 IDENT a
1:2 EQ ==
1:4 IDENT b
1:5 NEQ !=
1:7 IDENT c
1:8 LE <=
1:10 IDENT d
1:11 GE >=
1:13 IDENT e
1:14 LT <
1:15 IDENT f
1:16 GT >
1:17 IDENT g
1:18 ASSIGN =
1:19 NOT !
1:20 IDENT h
1:21 AND &&
1:23 IDENT i
1:24 OR ||
1:26 IDENT j
1:27 EQ ==
1:29 ASSIGN =
1:30 IDENT k
--- ERRORS ---
(none)
--- end ---
```

Fig. 8. Test 2.3

```
PS I:\BIE\3-2\Compiler\Tasks\02> .\task2 input.txt
1:1 IDENT x
1:3 ASSIGN =
1:5 IDENT y
1:9 IDENT z
1:13 IDENT w
1:14 SEMI ;
2:1 IDENT a
2:6 IDENT b
2:9 IDENT lives
--- ERRORS ---
1:7 INCOMPLETE_OPERATOR [&]
1:11 INCOMPLETE_OPERATOR [|]
2:5 INVALID_CHAR [?$]
2:9 INVALID_CHAR [9]
--- end ---
```

Fig. 9. Test 2.4

```
PS I:\BIE\3-2\Compiler\Tasks\02> .\task2 input.txt
1:1 KEYWORD while
1:7 IDENT i
1:9 LT <
1:11 IDENT n
1:13 LBRACE {
2:5 KEYWORD if
2:8 IDENT arr
2:11 LBRACKET [
2:12 IDENT i
2:13 RBRACKET ]
2:15 GE >=
2:18 IDENT max
2:22 AND &&
2:25 NOT !
2:26 IDENT found
2:32 LBRACE {
3:9 IDENT max
3:13 ASSIGN =
3:15 IDENT arr
3:18 LBRACKET [
3:19 IDENT i
3:20 RBRACKET ]
3:21 SEMI ;
4:5 RBRACE }
5:1 RBRACE }
--- ERRORS ---
(none)
--- end ---
```

Fig. 10. Test 2.5

4 Task 3: Literals and Comments

4.1 Design

The first character selects the rule: " string, ' character, @ annotation, : (COLON/SCOPE by peeking), . (up to three dots: DOT, RANGE, ELLIPSIS), - (ARROW only before >). A word starting with a letter or digit is read fully, then classified as HEX (0x) or BIN (0b). # skips the rest of the line.

4.2 Two-Buffer Scheme

The scanner buffers one line at a time. advance() and peek() move through it, and characters are copied into a separate lexeme string. The full two-buffer scheme with sentinels is not implemented.

4.3 Error Recovery

Errors are collected and printed after the tokens, each at the start of the bad lexeme: INVALID_CHAR, MALFORMED_NUMBER, UNTERMINATED_STRING, BAD_ESCAPE and BAD_CHAR_LITERAL.

Limitation. All tokens are handled *except multi-line block comments* (#[...]#), including nesting and UNTERMINATED_COMMENT. Tests 3.3 and 3.5 are affected.

4.4 Test Results

```
#include <iostream>
#include <cstring>
#include <string>
#include <fstream>
#include <vector>
#include <map>
#include <set>

using namespace std;

typedef struct invalid_char
{
    int row;
    int col;
    string ch;
    string name;
} inv;

map<string, string> op = {
    {"::", "SCOPE"},
    {":", "COLON"},
    {"->", "ARROW"},
    {".", "DOT"},
    {"..", "RANGE"},
    {"...", "ELLIPSIS"}
};

void skipws(string &prog, int &start)
{
    while (start < prog.length() && isspace(prog[start]))
    {
```

```
        ++start;
    }
}

string advance(string &prog, int &start, int count = 1)
{
    int pos = start;
    start += count;
    return prog.substr(pos, count);
}

char peek(string &prog, int &start)
{
    return prog[start + 1];
}

string readString(string &prog, int &start)
{
    string str = "";
    if (prog[start] != '"')
    {
        cerr << "Error: string does not start with '\"'\n";
        return "";
    }
    advance(prog, start); // read the first "

    while (start < prog.length() && prog[start] != '"')
    {
        str += advance(prog, start);
    }

    if (start >= prog.length())
    {
        cerr << "Error: unterminated string\n";
        return "";
    }

    advance(prog, start); /// read ending "
    return str;
}

char readChar(string &prog, int &start)
{
    char ch = prog[start + 1];
    advance(prog, start, 3);
    return ch;
}
int main(int argc, char *argv[])
{
    fstream inf(argv[1], ios::in);
```

```
if (!inf.is_open())
{
    return 0;
}

string prog;
string token_name;

int start = 0;
string token = "";

vector<inv> inv_char;

int row = 0;
int col = 1;
while (getline(inf, prog))
{
    token_name = "";
    start = 0;
    col = 1;

    row++;
    while (start < prog.length())
    {
        skipws(prog, start);

        switch (prog[start])
        {
        case '"':
            col = start + 1;
            token = readString(prog, start);
            token_name="STRING";
            break;
        case '\'':
            col = start + 1;
            if (peek(prog, start) == 'n') {
                token += advance(prog, start, 2);
            }
            else {
                token = string(1,readChar(prog, start));
            }
            token_name = "CHAR";
            break;
        case '@':
            token+=advance(prog, start);
            while (start < prog.length() && prog[start]!='@'
                    && !isspace(prog[start])) {
                token+= advance(prog, start);
            }
            token_name = "ANNOT";
```

```
        break;
    case ':':
        col = start + 1;
        if (peek(prog, start) == ':') {
             token += advance(prog, start);
        }
        token += advance(prog, start);
        token_name = op[token];
        break;
    case '.':
        col = start + 1;
        if (peek(prog, start) == prog[start])
        {
            if (prog[start+2] ==  prog[start]) {
                token += advance(prog, start, 3);
            }
            else {
                token += advance(prog, start, 2);
            }
        }
        else
        {
            token += advance(prog, start);
        }
        token_name = op[token];
        break;
    case '-':
        col = start + 1;
        if (peek(prog, start) == '>')
        {
            token += advance(prog, start, 2);
            token_name = op[token];
        }
        else
        {
            inv_char.push_back({row, start, string(1, prog[start]), "
                INVALID_CHAR"});
            start++;
        }
        break;
    default:
        col = start + 1;
        if (isalpha(prog[start]) || prog[start] == '0')
        {
            while (start < prog.length() && (isalpha(prog[start]) || (prog[start
                ] >= '0' && prog[start] <= '9')))
            {
                token += advance(prog, start);

            }
```

```
                if (token[0] == '0' && token[1] == 'x' && token.length() >= 3) {
                    token_name = "HEX";
                }
                if (token[0] == '0' && token[1] == 'b' && token.length() >= 3) {
                    token_name = "BIN";
                }
            }
            else
            {
                // # theke start kore shesh porjonto oi line e shb comment
                if (prog[start] == '#') {
                    if (prog[start+1]!='[') {
                        while (start < prog.length()) {
                            start++;
                        }
                    }

                }
                while (start < prog.length() && !isspace(prog[start]) && !isalpha(
                    prog[start]))
                {
                    token += advance(prog, start);
                }
                inv_char.push_back({row, start, token, "INVALID_CHAR"});
                token.clear();
            }
            break;
        }

        if (token != "")
            cout << row << ":" << col << "␣" << token_name << "␣" << token << '\n';

        token.clear();
        token_name.clear();
    }
}

cout << "---␣ERRORS␣---\n";
if (inv_char.size() == 0)
    cout << "(none)\n";
else
{
    for (auto it : inv_char)
    {
        cout << it.row << ":" << it.col + 1 << "␣" << it.name << "␣[" << it.ch << "]
            ␣\n";
    }
}

cout << "---␣end␣---\n";
```

```
    return 0;
}
```

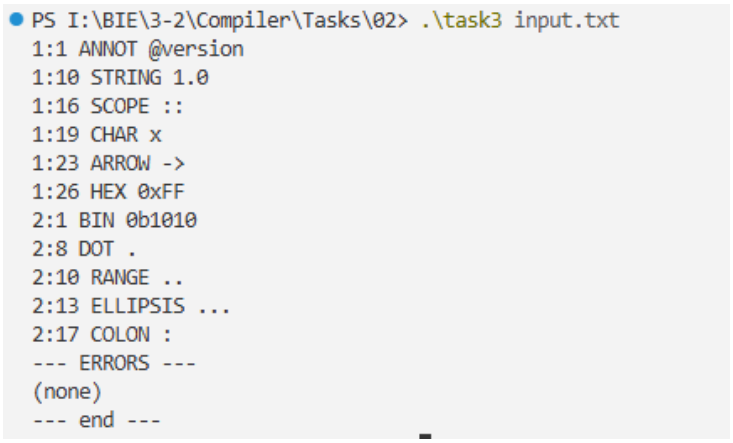


Fig. 11. Test 3.1

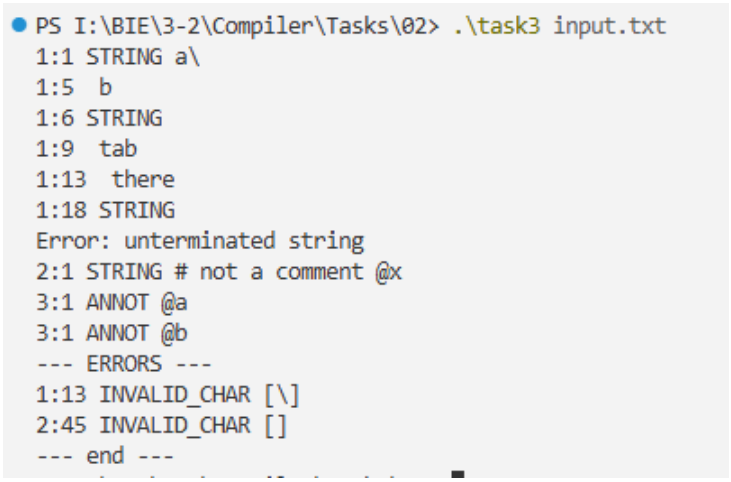


Fig. 12. Test 3.2

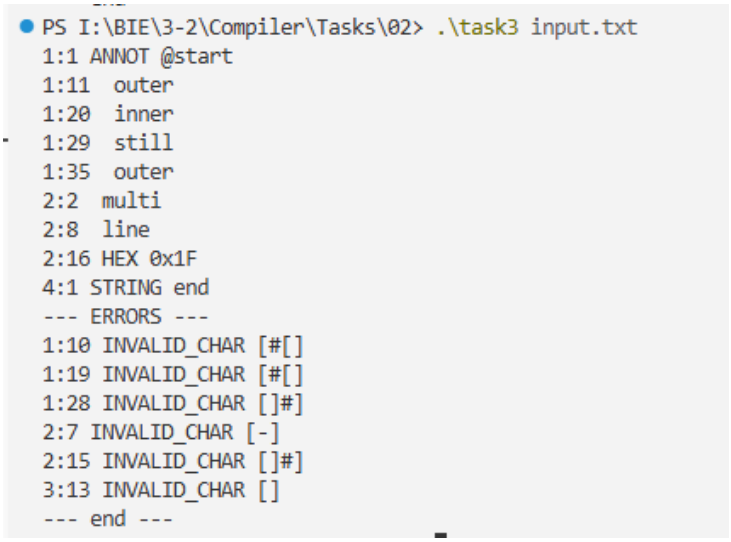


Fig. 13. Test 3.3 (block comment not supported)

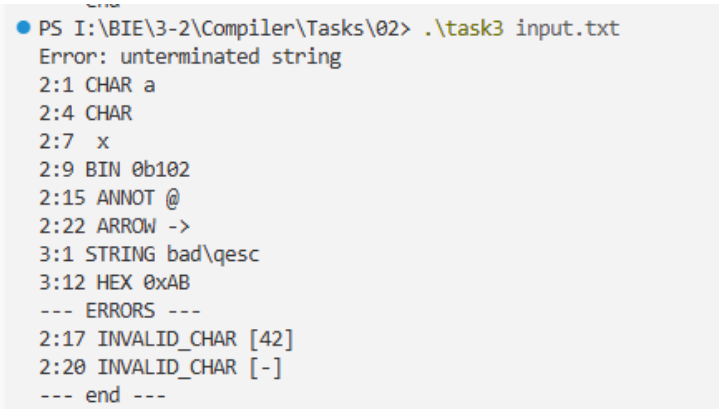


Fig. 14. Test 3.4

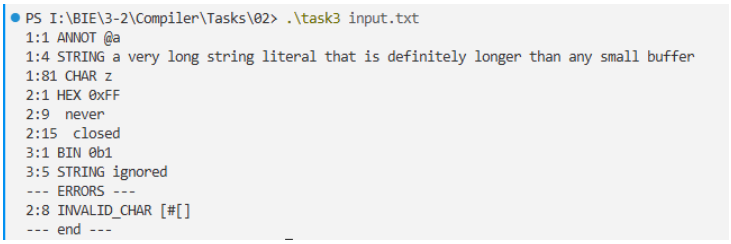


Fig. 15. Test 3.5 (block comment not supported)

5 Discussion

The hardest parts were lookahead for maximal munch (`3.14.5`, `===`, `0b102`) and deciding where an invalid run ends. Recording the column before consuming a lexeme fixed most position bugs. I would add nested block comments and the full two-buffer scheme.

6 Conclusion

Three tokenizers were built using `advance()`, `peek()`, maximal munch and non-stopping error recovery. Tasks 1 and 2 are complete; Task 3 handles all tokens except multi-line comments.